# ShifterSimulator — What This Project Does

---

## What Is This Project?

ShifterSimulator is a smart energy management platform built by **Decent Energy**. Its main job is to **automatically control home battery storage systems** to save customers money on their electricity bills and reduce their carbon footprint.

Every day, the system decides **when a customer's battery should charge and when it should discharge** — and then sends those instructions directly to the physical hardware installed in the customer's home.

---

## The Problem It Solves

Electricity prices in the UK change throughout the day. During off-peak hours (usually at night), electricity is cheap. During peak hours (morning and evening), it is expensive. Most homeowners with a battery don't know the best time to charge or discharge, so they either waste money or miss savings opportunities.

ShifterSimulator solves this by:

- Watching the electricity prices every day
- Looking at how much electricity the customer will likely use
- Checking how much solar energy their panels will generate
- Checking how clean or dirty the electricity grid is (carbon intensity)
- Then figuring out the best possible charge/discharge plan to minimize cost and carbon

---

## Who Uses It?

- **Customers** — homeowners with solar panels and battery storage who are on smart tariffs (like Octopus Agile). They receive a daily WhatsApp message summarizing tomorrow's battery plan.
- **Decent Energy team** — uses the system to monitor performance, onboard new customers, track savings, and manage the platform.
- **Physical inverter hardware** — the Shifter Service (a separate control system) reads the instructions generated here and actually sends commands to the battery.

---

## What Happens Every Day — The Big Picture

Here is the full story of what the system does each day, in plain language:

## 1. Collect fresh data (runs overnight)

Before any planning can happen, the system gathers all the information it needs:

- **Electricity prices** — fetched from Octopus Energy. Prices change every 30 minutes on Agile tariffs.
- **Carbon intensity** — fetched from the UK national grid API. Shows how much $CO_2$ is produced per unit of electricity at each time of day.
- **Inverter readings** — fetched from each customer's inverter cloud (Solis, FoxESS, GivEnergy, SolarEdge, or Eleven). This is the actual data about what the battery and solar panels did yesterday.
- **Smart meter readings** — fetched from Octopus. Shows how much electricity the customer actually consumed.

All of this is stored in a PostgreSQL database so it can be used quickly the next morning.

---

## 2. Generate tomorrow's battery plan (runs every morning)

This is the heart of the system. For every active customer, it does the following:

**a) Fetch the customer's setup** The system reads each customer's configuration from the Accounts API — what inverter they have, how big their battery is, what electricity tariff they're on, where they live, whether they have solar panels, and what their minimum battery level must be.

**b) Fetch what electricity they will likely use tomorrow** A consumption forecast (predicted kWh per 30-minute slot) is pulled from the database. This was generated separately by a machine learning forecasting model.

**c) Fetch tomorrow's prices and carbon data** Half-hourly electricity prices and carbon intensity values are pulled from the database for tomorrow's 48 slots.

**d) Fetch tomorrow's solar forecast** If the customer has solar panels, a calibrated solar generation forecast is pulled from the database (powered by Solcast irradiance data).

**e) Decide the best battery schedule** Now the system runs an optimization algorithm. It looks at all 48 half-hour slots of the day and asks: "Given the prices, the expected consumption, the solar generation, and the battery size — what is the cheapest AND cleanest plan?"

The algorithm works as follows:

- It balances two goals: **minimize electricity cost** (80% weight by default) and **minimize carbon emissions** (20% weight by default).
- Customers can adjust this balance — a "pure green" customer might choose 100% carbon focus; a cost-focused customer might choose 100% price focus.
- The algorithm tries every possible combination of charge/discharge decisions across the day and finds the one that results in the lowest total cost/carbon.
- It respects physical constraints: the battery cannot go below its minimum safe level, the inverter cannot charge faster than its rated power, and during peak hours the battery must

hold enough charge to cover the evening's demand.

The output is a plan like:

- 01:00–03:00 → Charge battery (cheap overnight electricity)
- 07:00–09:00 → Use battery (avoid expensive morning peak)
- 11:00–14:00 → Allow solar (let solar panels charge battery and cover load)
- 17:00–20:00 → Use battery (avoid expensive evening peak)
- Everything else → Do nothing (buy from grid at normal rate)

**f) Save the plan to the database** The full plan — one row per 30-minute slot with the action, expected cost, expected consumption, expected carbon — is saved to the database. It can be queried and analysed later to measure actual savings.

---

## 3. Send the plan to the hardware

The generated plan is uploaded to the **Shifter Service** — a separate control-plane system that interfaces directly with physical inverters. The Shifter Service stores the instructions in MongoDB. The inverter controller running in the customer's home periodically checks MongoDB and executes the instructions (charge at 01:00, discharge at 17:00, etc.).

---

## 4. Notify the customer

A **WhatsApp message** is sent to the customer (via Twilio) with a plain-English summary of tomorrow's plan. It tells them:

- How many charge cycles are planned and what the average cost per kWh will be
- How many discharge cycles are planned and what the average saving per kWh will be
- The timing of the first few actions

This is sent once per day per customer, regardless of how many times the pipeline runs.

---

## 5. Monitor and report (runs throughout the day)

Separate monitoring scripts run to keep the team informed:

- **Accuracy reporting** — compares what the system predicted vs what actually happened. Shows how much money customers actually saved.
- **SOC deviation check** — checks whether the battery's actual charge level is following the plan. A big difference could mean the inverter isn't responding to commands.
- **Carbon data health check** — confirms that carbon intensity data is available and looks correct for the next 48 hours.
- **Price anomaly checks** — alerts the team if any electricity prices look unusually high or low

(could be a data error or a genuine market event).

- **New customer signups —** notifies the team when new customers are onboarded and the system starts controlling their batteries.

---

## Special Cases the System Handles

**New customers —** When a customer signs up for the first time, the system sends them a welcome WhatsApp message and an onboarding email, letting them know the system has started managing their battery.

**Multiple inverter brands —** The system works with 5 different inverter brands (Solis, FoxESS, GivEnergy, SolarEdge, Eleven). Each has its own cloud API with different authentication and data formats. The system handles all of them.

**Simple vs smart mode —** Customers on simple flat-rate tariffs (where there is no price variation throughout the day) are handled with a simpler rule-based approach: charge during off-peak hours, discharge during peak hours. Customers on dynamic tariffs like Agile get the full optimization algorithm.

**Fallback logic —** If the smart optimization algorithm fails for any reason, the system automatically falls back to the simpler rule-based approach so the customer's battery is never left without a plan.

**Staging vs production —** The system has a staging environment for testing. In staging, no WhatsApp messages are sent and no commands are sent to real hardware. This prevents accidental interactions with live customer equipment.

**Battery degradation —** Over time, batteries lose capacity. The system tracks battery capacity history and uses the correct current capacity when generating the schedule, rather than assuming the original specification.

---

## What the REST API Does

The system also runs a small web API that other internal tools can call to fetch historical inverter data. Given a customer ID and a date range, it returns the half-hourly telemetry (solar generation, battery charge/discharge, grid import/export) for that period. This is used for dashboards, savings reports, and customer-facing tools.

---

## Key Numbers

| Thing | Detail |
|---|---|
| Time resolution | 30-minute slots (48 per day) |
| Optimization weight | 80% price, 20% carbon (default, adjustable per customer) |
| Supported inverter brands | Solis, FoxESS, GivEnergy, SolarEdge, Eleven |

| Thing | Detail |
| --- | --- |
| Messaging platform | WhatsApp via Twilio |
| Primary database | PostgreSQL |
| Instruction storage | MongoDB (via Shifter Service) |
| Notification on errors | Slack |
| Error tracking | Sentry |

---

## In One Sentence

ShifterSimulator watches electricity prices and carbon data every day, figures out the best possible battery charge/discharge schedule for each customer, sends those instructions to their home hardware, and notifies the customer — all automatically, every single day.